Studio Nx

JS 15 — initDuplicateChecks()

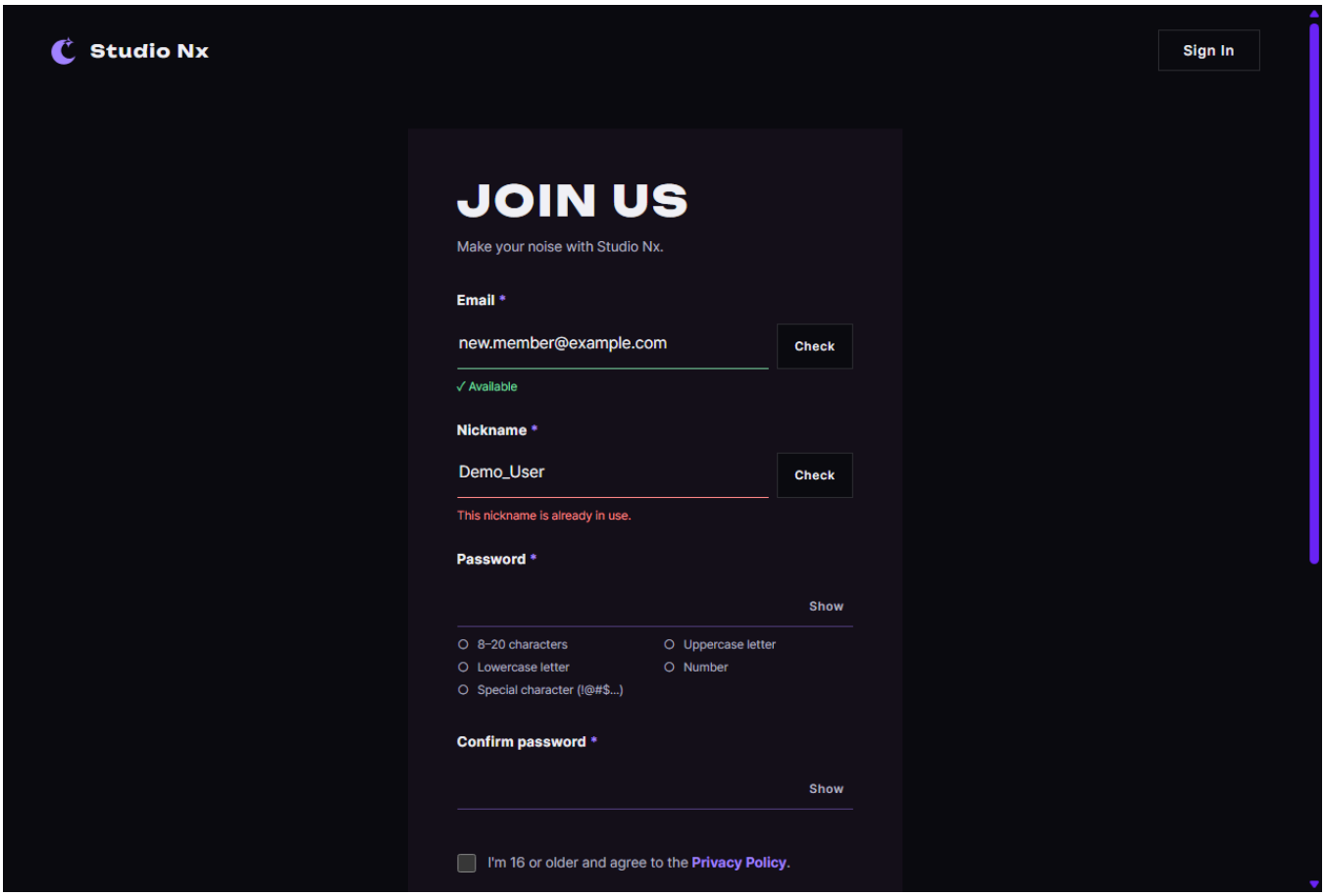
2026-10-12

JS 15 — `initDuplicateChecks()`

항목	내용
파일	`studionx/frontend/static/js/pages/join.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-101 실시간 중복 확인, FR-108 탈퇴 후 3일)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	14 `initLoginModalFromHash()` · 16 `initPasswordRules()`

1. 하는 일

join.html의 이메일 칸과 닉네임 칸에 입력한 값이 **이미 쓰이고 있는지** 서버에 물어서 칸 아래에 보여 줍니다. **Check** 버튼을 누르거나, 입력을 멈추고 0.5초가 지나면 묻습니다.



입력	칸 아래
(빈 칸), `abc`, `kim@naver`	빨강 "Enter a valid email address."이고, **서버에 묻지 않습니다**
없는 이메일	잠깐 "Checking..."이 나온 뒤 초록 "✓ Available"이 나옵니다
가입된 이메일 (`DEMO@Example.com`처럼 대소문자가 달라도)	빨강 "This email is already in use."
탈퇴한 지 3일이 안 된 이메일	빨강 "You can sign up again after Oct 9." (날짜는 **사용자의 시간대** 기준)
닉네임 1자, 21자	빨강 "Use 2–20 characters."이고, 서버에 묻지 않습니다

입력	칸 아래
가입된 닉네임 (`Demo_User` ↔ `demo_user`)	빨강 "This nickname is already in use."

입력을 고치면 결과가 바로 지워집니다. 다시 확인해야 가입할 수 있습니다.

2. 언제 쓰나

join.html이 열릴 때 `join.js`가 한 번 부릅니다. 묻는 곳은 P1-04에서 만든 API입니다.

칸	API	서버
이메일	`GET /api/v1/auth/check-email?email=`	백엔드 23
닉네임	`GET /api/v1/auth/check-nickname?nickname=`	백엔드 24

확인 결과는 가입 전송(17번 `initJoinForm`)이 씁니다. "✓ Available"이 나온 값은 칸에 `data-checked="그 값"`으로 남습니다. 가입 버튼을 누를 때 지금 값과 같은지 보고, 다르면(확인하지 않았거나 고친 뒤라면) 가입을 막습니다.

3. 코드

import (`join.js` 맨 위)

```
import { initLoginForm, initPasswordToggles, initOAuthPlaceholders, setFieldState } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { api } from '../core/api.js';
```

`api` 줄을 이번에 추가했습니다. 이 함수는 `api`(서버에 묻기)와 09번 `setFieldState`(칸 상태)를 씁니다.

함수

```
/** 중복 확인 칸마다 다른 것 — 주소 · 문구 · 서버에 묻기 전 형식 검사 */
const CHECKS = {
  email: {
    path: '/auth/check-email?email=',
    taken: 'This email is already in use.',
    invalid: 'Enter a valid email address.',
    isValid: (input) => input.value.trim() !== '' && input.checkValidity(),
  },
  nickname: {
    path: '/auth/check-nickname?nickname=',
    taken: 'This nickname is already in use.',
    invalid: 'Use 2–20 characters.',
    isValid: (input) => input.value.trim().length >= 2 && input.value.trim().length <= 20,
  },
};

export function initDuplicateChecks() {
  for (const kind of Object.keys(CHECKS)) {
    const rule = CHECKS[kind];
    const field = document.getElementById(`join-${kind}-field`);
    const input = document.getElementById(`join-${kind}`);
```

```
    const button = document.querySelector(`[data-check="${kind}"]`);
    if (!field || !input) continue;
    let timer = null;

    const check = async () => {
      clearTimeout(timer);
      delete field.dataset.checked; // 이번 확인을 통과해야 다시 남김
      const value = input.value.trim();
      if (!rule.isValid(input)) {
        setFieldState(field, 'error', rule.invalid);
        return;
      }
      setFieldState(field, null, 'Checking…');
      let result;
      try {
        result = await api.get(rule.path + encodeURIComponent(value), { auth: false });
      } catch (err) {
        if (input.value.trim() !== value) return; // 그사이 입력이 바뀜 — 옛 응답은 버림
        setFieldState(field, 'error', err.status === 422 ? rule.invalid : err.message);
        return;
      }
      if (input.value.trim() !== value) return;
      if (result.available) {
        field.dataset.checked = value;
        setFieldState(field, 'ok', '✓ Available');
      } else if (result.reason === 'REJOIN_TOO_SOON') {
        const day = new Date(result.available_at).toLocaleDateString('en-US', { month: 'short', day: 'nu
meric' });
        setFieldState(field, 'error', `You can sign up again after ${day}.`);
      } else {
        setFieldState(field, 'error', rule.taken);
      }
    };

    input.addEventListener('input', () => {
      delete field.dataset.checked; // 값이 바뀌면 다시 확인해야 함
      setFieldState(field, null);
      clearTimeout(timer);
      if (rule.isValid(input)) timer = setTimeout(check, 500);
    });
    if (button) button.addEventListener('click', check);
  }
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다. 바뀌는 부분만 적었습니다)

```
for (const [kind, rule] of Object.entries(CHECKS)) { … }
…
const { available, reason, available_at } = await api.get(…);
…
button?.addEventListener('click', check);
```

최신 문법	실제 코드	뜻
`for (const [kind, rule] of Object.entries(CHECKS))`	`for (const kind of Object.keys(CHECKS))` + `const rule = CHECKS[kind]`	열쇠와 값을 한 번에 꺼냅니다

최신 문법	실제 코드	뜻
`const { available, reason, available_at } = await …`	`result = await …` + `result.available`	응답에서 칸을 꺼냅니다 (구조 분해)
`button?.addEventListener(…)`	`if (button) button.addEventListener(…)`	버튼이 있을 때만 실행합니다

문법 풀이

import 문법	뜻
`import { api } from '../core/api.js'`	`pages/`에서 한 폴더 위로 올라가 `core/api.js`를 가리킵니다

식	뜻
`` `join-${kind}-field` ``	`kind`가 `'email'`이면 `'join-email-field'`가 됩니다
`input.checkValidity()`	브라우저의 기본 검사(`type="email"`, `required`, `maxlength`)를 통과하면 `true`입니다
`setTimeout(check, 500)`, `clearTimeout(timer)`	0.5초 뒤에 실행하도록 예약하고, 예약을 취소합니다. 입력할 때마다 취소하고 다시 예약하므로, **입력을 멈춘 뒤** 0.5초가 지나야 실행됩니다
`encodeURIComponent(value)`	주소에 넣을 수 있게 바꿉니다 (`+` → `%2B`, `@` → `%40`). `a+b@x.com`의 `+`가 공백으로 바뀌는 것을 막습니다
`delete field.dataset.checked`	`data-checked` 속성을 지웁니다
`new Date(…).toLocaleDateString('en-US', { month: 'short', day: 'numeric' })`	UTC 시각을 **이 컴퓨터의 시간대**로 바꿔서 `Oct 9` 모양으로 만듭니다

4. 흐름

Check를 누를 때 (또는 자동 확인)

1. 예약된 자동 확인을 취소하고, 지난번 통과 표시를 지웁니다.
2. 형식이 틀리면 빨간 문구를 보여 주고 끝냅니다 (서버에 묻지 않습니다).
3. "Checking…"을 보여 주고 서버에 묻습니다.
4. 응답이 오기 전에 입력이 바뀌었으면 응답을 **버립니다**.
5. 사용할 수 있으면 초록 문구와 `data-checked`를, 탈퇴한 지 3일이 안 되었으면 날짜 문구를, 사용 중이면 빨간 문구를 보여 줍니다.

입력할 때

1. 지난 결과를 지웁니다.
2. 형식이 맞으면 0.5초 뒤에 자동 확인을 하도록 예약합니다.

5. 왜 이렇게

선택	이유
서버에 묻기 전에 형식을 검사	`abc`처럼 틀린 값은 서버까지 보낼 필요가 없습니다. 서버도 422로 막지만(23, 24), 요청을 아낄 수 있습니다
자동 확인은 형식이 맞을 때만	`kim@na`까지 쳤을 때 빨간 "Enter a valid…"가 뜨면 입력 중인 사람을 재촉하는 셈입니다. 그래서 다 쳤을 때만 묻습니다. Check 버튼을 누르면 형식 문구도 보여 줍니다

선택	이유
0.5초 기다림 (템플릿 권장)	글자마다 묻지 않습니다. `kim@naver.com` 13자를 칠 때 요청이 13번에서 1번으로 줄어듭니다
늦게 온 응답은 버림	`demo@…`를 확인하는 중에 다른 값으로 고치면, 늦게 온 "사용 중" 결과가 새 값에 붙을 수 있습니다. 그래서 응답이 왔을 때 칸의 값이 물어본 값과 같은지 확인합니다
확인할 때마다 `data-checked`를 먼저 지움	시험에서 찾은 빈틈입니다. 통과한 뒤 (입력 이벤트 없이) 값이 바뀌면, "사용 중"이 나와도 옛 통과 표시가 남아 있었습니다. 이제는 이번 확인을 통과해야만 표시가 남습니다
날짜는 사용자의 시간대로	서버는 UTC(`…Z`)로 줍니다. 한국 사용자에게는 한국 날짜로 보여야 합니다
칸마다 다른 부분은 `CHECKS` 표로 정리	이메일과 닉네임은 동작이 같고, 주소, 문구, 형식 규칙만 다릅니다. 같은 코드를 두 번 쓰지 않기 위해서입니다

6. 시험 (실제 join.html에서 실제 서버로, 15개)

데모 계정(`demo@example.com`, `demo_user`)과 시험용으로 **탈퇴한 지 1일 된 계정**을 DB에 넣고 확인했습니다 (탈퇴 계정은 끝난 뒤 지웠습니다).

시험	확인한 것
빈 칸, `abc` + Check	"Enter a valid email address."가 나오고, 서버 요청은 0번입니다
없는 이메일	누르자마자 "Checking…"이 나온 뒤 "✓ Available"이 나오고, `data-checked`가 남습니다
`DEMO@Example.com`	"This email is already in use."가 나오고, `data-checked`가 없습니다
탈퇴한 지 1일 된 이메일	"You can sign up again after Oct 9."
입력	결과가 바로 지워지고, 0.3초에는 아직 묻지 않으며, 0.5초가 지나면 자동으로 확인해서 ✓가 나옵니다
`auto.ch`처럼 덜 친 입력	자동으로 확인하지 않습니다 (빨간 글 없음, 요청 0번)
확인 중에 값을 바꿈	옛 값의 "사용 중" 결과가 새 값에 붙지 않습니다
닉네임 `a`, `DEMO_USER`, `" fresh_nick "`, 탈퇴한 지 1일 된 닉네임	차례대로 2–20자 문구, 사용 중, ✓(`data-checked="fresh_nick"`), 재가입 날짜 문구가 나옵니다

처음 돌렸을 때는 14/15였습니다. 위 §5의 빈틈(`data-checked`가 남음)을 고친 뒤 15개 모두 통과했습니다.